



Concepts of Data Science

Bloom filter

Arnoud Vandeveld, Valery Atemefac

May 19, 2026

Contents

1	Bloom filter theory	3
1.1	Definition	3
1.2	False positive rate	3
1.3	Time and space complexity	4
2	Hash functions	5
2.1	Examples of hash functions	5
2.1.1	lose lose	5
2.1.2	djb2	5
2.1.3	sdbm	5
2.1.4	MurmurHash3	6
2.2	Kirsch-Mitzenmacher-Optimization	6
2.3	Distribution of hash values	6
2.4	Correlation between hash functions	8
3	Empirical analysis	11
3.1	Optimal number of hash functions	11
3.2	Convergence of false positive rate	11
3.3	Effect of increasing filter size on false positive rate	12
3.4	Time analysis	13
3.4.1	Exploratory analysis	13
3.4.2	Effect of hash functions	15
3.4.3	Effect of increasing string length	16
4	Conclusion	18
5	Code & Implementation	18
6	References	18

1 Bloom filter theory

1.1 Definition

A Bloom filter is a probabilistic data structure that tests if an element, in most cases strings, is part of a set. It is probabilistic in the sense that when testing for a specific element we either know if it is definitively not in the set or possibly in the set, we thus can't with hundred percent say if an element is part of the bloom filter if it is actually contained in it. A Bloom filter is specifically a bit array, i.e. a list that only contains 0's or 1's. To construct a Bloom filter we first have to specify size of the filter, this is usually denoted by m . An empty bloom filter is now defined as an array of length m where all the bits are set to 0. To now add an element to the Bloom filter we apply k different hashing functions to the to be added element, which result in a series of k indices. All of these indices are now set to 1 in the Bloom filter which indicate that the element was added. To test whether an element is in the set we compute the hash functions and see if all of the indices are set to one, this indicates that the element is possibly in the set. If one of the indices is 0 we definitely know that it isn't a part of the set. The reason why we can't definitively say that an element is part of the set is because of the fact that a hash function of another element might set the same index also to one. An element that a Bloom filter recognizes as a part of the set while it is actually not is referred to as a *false positive*.

For example let's say we have a rudimentary Bloom filter of size 5 with 3 hashing functions h_1, h_2 and h_3 that maps an element a to 1, 2 and 3 respectively; the Bloom filter would now be of the form $[1, 1, 1, 0, 0]$. By adding another element b with $h_1(b) = 2, h_2(b) = 3$ and $h_3(b)$ results in the bit array of the form $[1, 1, 1, 1, 0]$. If there is now an element c that has $h_1(c) = 1, h_2(c) = 3$ and $h_3(c) = 4$ we would get the exact same array, so from this bit array we do not know if c is present. Since c wasn't originally added in our case it is seen as a false positive. Even in a filter with only one element a we can't test that a is in the filter because if $h_1(a) = 1, h_2(a) = 2, h_3(a) = 3$ and $h_1(b) = 3, h_2(b) = 2, h_3(b) = 1$ we can't differentiate between a and b .

1.2 False positive rate

Define ε as the probability of a false positive in the Bloom filter. Assume that we have a bloom filter of size m with k independent hash functions that are uniformly distributed. The probability of a bit being not turned into a '1' by a particular hash function is now thus $1 - \frac{1}{m}$. For m independent hash functions this turns into a probability of $(1 - \frac{1}{m})^k$. Since $e^{-1} = \lim_{m \rightarrow \infty} (1 - \frac{1}{m})^m$, this now means that for m large enough: $(1 - \frac{1}{m})^k = ((1 - \frac{1}{m})^m)^{\frac{k}{m}} \approx e^{-\frac{k}{m}}$. This last equation is now approximately the probability that a certain bit is still 0 after one insertion into the bloom filter. If we inserted n elements into the bloom filter the probability of this bit stay 0 thus becomes roughly $e^{(-\frac{k}{m})n} = e^{-\frac{kn}{m}}$. The probability now of this bit becoming 1 is then easily approximated by $1 - e^{-\frac{kn}{m}}$.

Now to test if an element x is a false positive, while knowing it is not in the set, we have to check if the bits at positions $h_1(x), \dots, h_k(x)$ are all ones. It is worth noting that we now assume that $h_1(x), \dots, h_k(x)$ all give different indices. We can assume this since all of our hashing functions are independent and because the probability that two or more bits align is negligible if m is much greater than k . Generally now the probability of a false positive is given by $\varepsilon \approx (1 - e^{-\frac{kn}{m}})^k$.

We can now minimize this false positive rate in regards to k for fixed values of m and n . Minimizing in regards to m and n makes no sense as, $m \rightarrow \infty$ and $n \rightarrow 0$ implies that the false positive rate converges to zero. This result is logical since adding more bits and decreasing the amount of entered words decreases the chance that two possible bits overlap. To find the (roughly) optimal k we solve the following equation:

$$\frac{d}{dk} (1 - e^{-\frac{kn}{m}})^k = 0 \quad (1)$$

This results in $k = \frac{m}{n} \ln(2)$, this is a minimum because the double derivative in that point is positive. Since this number is in most cases non-natural, we choose the $\tilde{k} \in \{[k], \lceil k \rceil\}$ such that $(1 - e^{-\frac{\tilde{k}n}{m}})^{\tilde{k}}$ is minimal.

1.3 Time and space complexity

We first look at the time complexity for searching and adding an element in a Bloom filter.

The time complexity for adding an element x to the filter we have to calculate $h_i(x)$ for all k hash functions. For a fixed element x the calculation of the hash values for a hash functions takes $O(1)$ or constant time. Modifying the value of an element at a given index in a list can be done in $O(1)$ time, since we have k of these indices the insertion process (turning zeroes into ones) takes $O(k)$ time. The time complexity is thus now $O(k)$.

In general the amount of time needed for searching for an element x is lower than adding that same element. This is because if the bit at index $h_i(x)$ is equal to zero than the element is definitely not in the Bloom filter, there is thus no need to calculate $h_{i+1}(x), h_{i+2}(x), \dots, h_k(x)$. In worst case we still have to calculate k hash values, this happens if the element was added to the Bloom filter or if it is a false positive. From this the worst case time complexity is given by $O(k)$, which is the same as adding an element.

It is worth noting that for both of these time complexities we viewed the element x as constant, this reduces the time complexity of a hash function to $O(1)$. If x is now variable the time complexity of a hash function h_i is given by $O(f_i(l))$, in which l denotes the length of the string and $f_i(l)$ some function. Most hash functions, including the ones we will see in Section 2, have a time complexity of $O(l)$ due to iterating over each character in the string. Now if all of the ever increasing k hash functions have this time complexity, the time complexity for adding and searching an element is than given by $O(kl)$.

The space complexity of the bloom filter is $O(m)$, since m is defined as the amount of bits needed.

2 Hash functions

Before giving examples of hash functions we remark that the range of hash functions is usually all of the natural numbers, this naturally creates a problem if the size of the bloom filter is smaller than the the output of a hash function for a particular element. We can circumvent this problem by using $h(x) \bmod m$ instead of $h(x)$ for all elements and/or strings x and hash function h .

2.1 Examples of hash functions

2.1.1 lose lose

The lose lose hash function is the simplest hash function. It returns the sum of the ASCII-values of each character in the string and can be implemented with the following Python code:

```
1 def lose_lose(s: str) -> int:
2     """Implementation of loselose hash function."""
3     h = 0
4     for char in s:
5         h += ord(char)
6     return h
```

The main drawback of the lose lose hash function in regards to bloom filters is that strings of the same length which contain the same characters are mapped to the same values. This increases to false positive rate drastically. This is especially relevant if the data is for example DNA sequences or numbers. It is also worth noting that the output of the lose lose function is largely due to the amount of characters in the string. If we now only enter strings of a maximal length, we expect to reach an upper bound that may be substantially smaller than the Bloom filter size m . This means that a large chunk of the bits at the backend of the array are largely untouched. This consequence also increases the false positive rate.

2.1.2 djb2

The djb2 hash function is simple hash function that updates the hash for each character in the input string. The steps in the function are given in the following Python code:

```
1 def djb2(s: str) -> int:
2     """Implementation of the djb2 hash function."""
3     h = 5381
4     for char in s:
5         h = ((h << 5) + h) + ord(char) # h = h * 33 + ord(char)
6     return h
```

We use $(h \ll 5) + h$, in which $h \ll 5$ denotes shifting 5 bits to the left, instead of $h \cdot 33$ because $h \cdot 33 = h \cdot 32 + h$ and because a multiplication with a power of two is just a bit shift. This implementation is now more efficient because nothing has to be calculated with a bit shift.

A potential downside of djb2 is that the hash values increase drastically for the amount of characters in a string. For example $djb2(abc) = 193485963$.

2.1.3 sdbm

The sdbm hash function, like the djb2 hash function, is also a simple hash function that updates the hash for each character in the input string. The steps in the function are given in the following Python code:

```
1 def sdbm(s: str) -> int:
2     """Implementation of the sdbm hash function."""
3     h = 0
4     for char in s:
5         h = ord(char) + (h << 6) + (h << 16) - h
6     return h
```

A potential downside of sdbm is that the hash values increase drastically for the amount of characters in a string, like djb2. For example $sdbm(abc) = 417419622498$.

2.1.4 MurmurHash3

MurmurHash3, abbreviated as mmh3, is a more complicated hash function that sidesteps the issue of massive hash values which are received in djb2 and sdbm. MurmurHash3 is also dependent on a seed which allows us to easily construct multiple hash functions which are independent. For our current implementation we will always use a default seed of 815. Since the algorithm behind MurmurHash3 function is quite complex, it falls out the scope of this report.

2.2 Kirsch-Mitzenmacher-Optimization

One main drawback of implementing a Bloom filter is the defining and constructing for a series of k hash functions, this is especially prominent if the optimal number of hash functions, which is given by $\frac{m}{n} \ln(2)$, is extraordinarily high. In these cases we do not have simple enough basic hash functions to apply to our Bloom filter. In these cases we, so far, have to rely on more complicated hash functions with different set seeds like MurmurHash. Luckily there is a way to circumvent this problem; it is showed in [1] by Adam Kirsch and Michael Mitzenmacher that from two base independent and uniformly distributed hash functions h_1 and h_2 we can construct the following k hash functions with $k \in \mathbb{N}_0$:

$$g_i(x) = h_1(x) + ih_2(x), \quad \forall i \in \{0, 1, \dots, k\} \quad (2)$$

for which the asymptotic false positive rate for these k hash functions is similar as compared to the use of k independent and uniformly distributed hash functions. This optimization allows us to for example construct unlimited hash functions from djb2 and sdbm.

2.3 Distribution of hash values

We will now look at the mean and median hash values for each of the four described hash functions for two datatypes; English words and DNA sequences.

For English words we will use the dataset from [2] which contains 479 thousand English words in a txt file. For DNA sequences we will look at all the DNA sequences of utmost 10 nucleotides. This data was manually created and contains roughly 1.4 million sequences.

	lose_lose	djb2	sdbm	mmh3
Mean	1001	$2.485 \cdot 10^{66}$	$2.108 \cdot 10^{208}$	2149443987
Median	974	$2.499 \cdot 10^{17}$	$3.841 \cdot 10^{40}$	2150490743

Table 1: Mean and median hash values of English words for each hash function.

	lose_lose	djb2	sdbm	mmh3
Mean	694	$6.231 \cdot 10^{18}$	$1.211 \cdot 10^{45}$	2146922627
Median	706	$8.245 \cdot 10^{18}$	$1.507 \cdot 10^{45}$	2145824059

Table 2: Mean and median hash values of DNA sequences for each hash function.

From the tables we can see that the hashes of the lose_lose function are magnitudes smaller than the other hash functions. This makes sense since we only add the order of the characters, which usually are values round 100, and since we only look at strings of a short-length (DNA sequences with a maximal length of 10 and most English words are shorter than 10 letters). For djb2 and especially sdbm we have very large values, this is due to the iterative nature of the functions where we multiply the previous hash value.

We will now look at the distribution of these hash functions after they are modulo reduced by a

number m . This is relevant since if m is the size of the bloom filter, then the histogram shows how uniformly the values are distributed across the bloom filter. It is important to check this since for our derivation of the false positive rate we assumed that the hash functions were uniformly distributed. For the English words we will use $m = 2 \cdot 10^5$, which is roughly 40% of the amount of English words. For the DNA sequences we will use $m = 7 \cdot 10^5$ which is roughly half of the amount of DNA sequences. The relevant histograms, which approximate the distribution, are shown below.

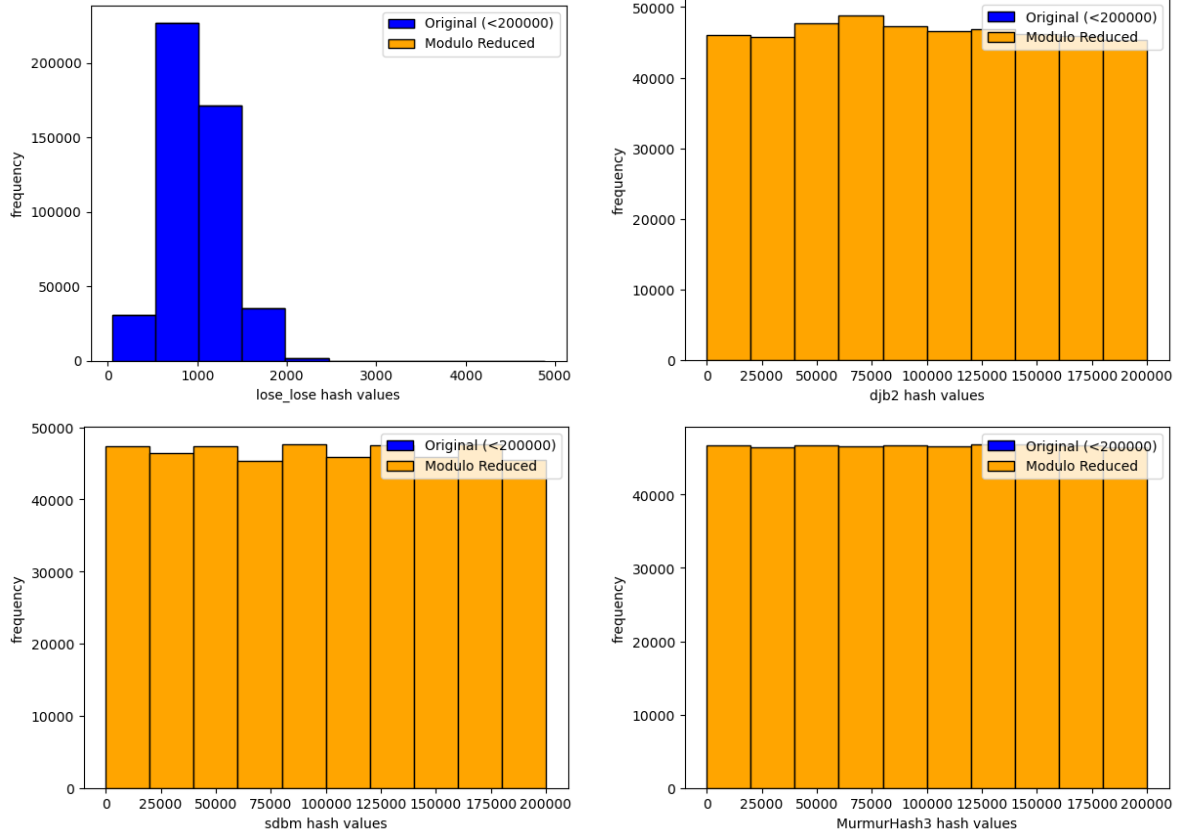


Figure 1: Distribution of hash values of English words for multiple hash functions.

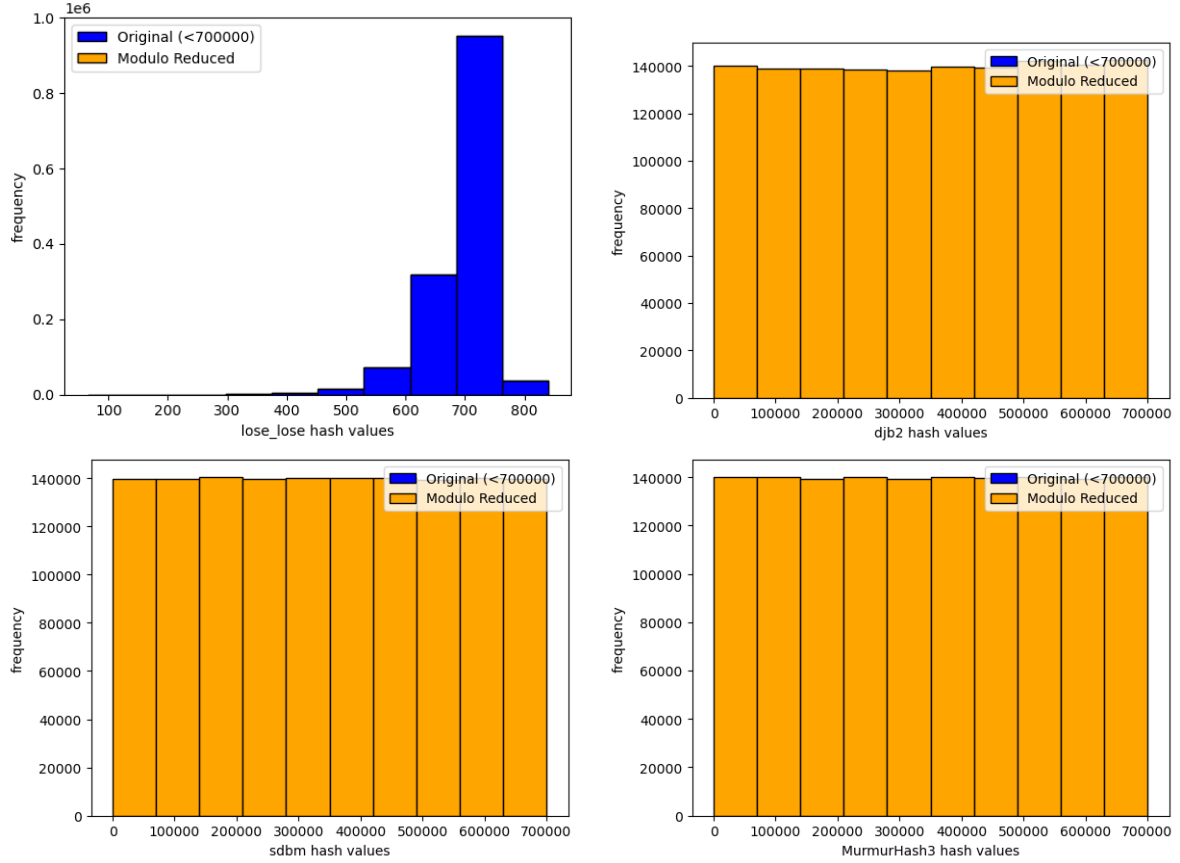


Figure 2: Distribution of hash values of DNA sequences of at most length 10 for multiple hash functions.

From the histograms we can see that besides the `lose_lose` hash function that practically each hash value gets modulo reduced. This result is logical because as seen in tables 1 and 2 these hash values are magnitudes greater than the chosen Bloom filter sizes.

From the histogram we can also see that every hash function besides `lose_lose` results in roughly a uniform distribution of hash values. This result reinforces our belief that `djb2`, `sdbm` and `mmh3` are good hash functions and that `lose_lose` is an intentionally terrible hashing function. This can also be seen in the upper bound of the histograms for the English words and the DNA sequences, which is 5000 and 800 respectively. This implies that most of the hash values get inserted in the beginning part of the Bloom filter and that the majority of the array isn't utilized.

Modifying the m values, which isn't given here, gives roughly the same distributions as shown here and thus show no deviations. This can be manually checked by going to the GitHub repository given in Section 5 and modifying the `m1` and `m2` values in the script `'HistogramHashFunctions.py'` under the folder `'examples'`.

2.4 Correlation between hash functions

Besides being uniformly distributed the hash functions should also be uncorrelated with each other to assume that the false positive rate is (roughly) of the form $(1 - e^{-\frac{kn}{m}})^k$. We now give scatter plots of the hash values for each combination hash functions for both of the datasets, English words and DNA, to see how correlated they are. For all of these plots we use the same values in m as given in 2.3.

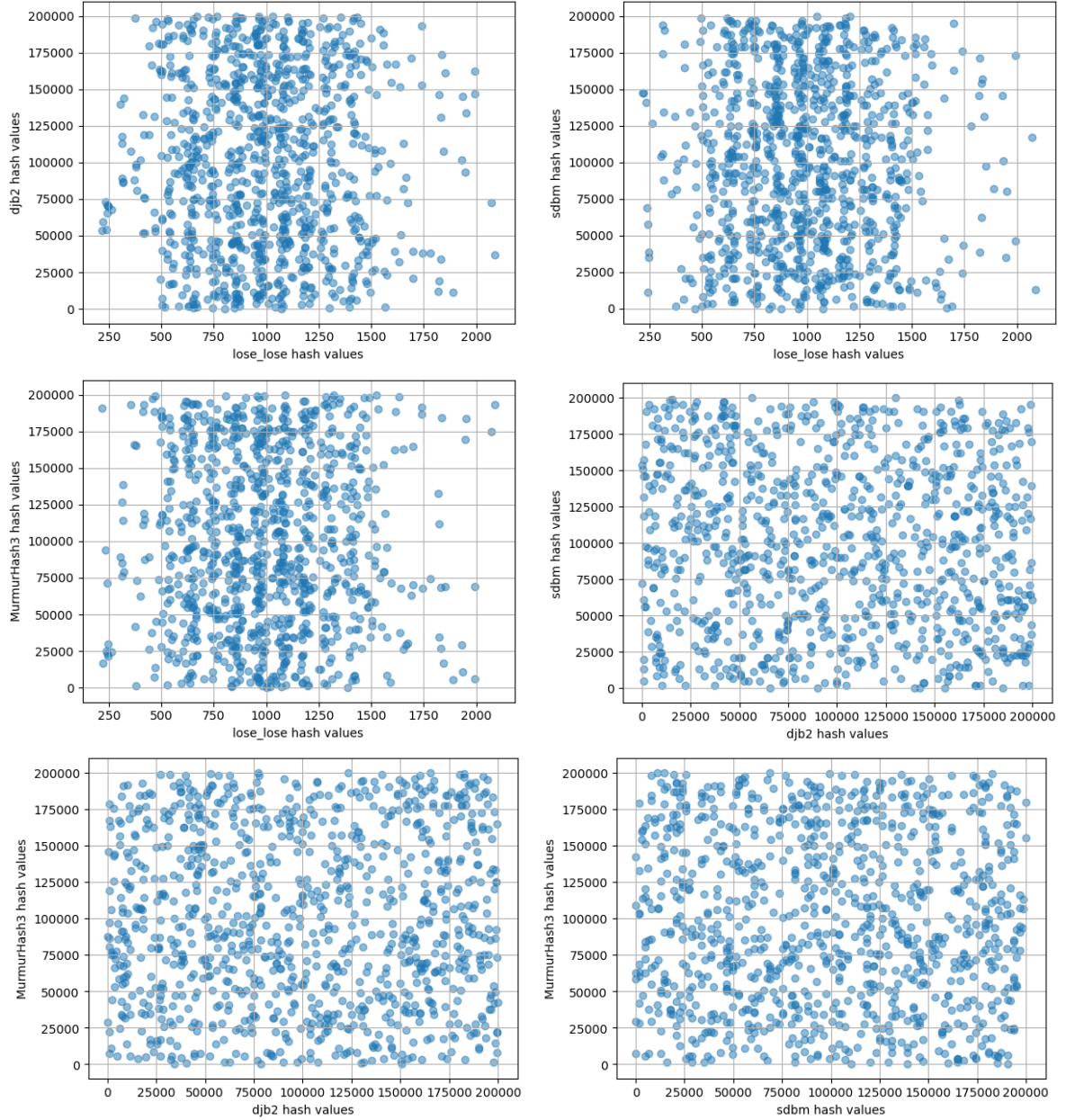


Figure 3: Correlation plots of each hash function combination for a dataset of English words.

In general there does not seem to be any correlation between any of the hash functions. The plots for the `lose_lose` hash function is more squeezed but this is due to the distribution of the hash values being heavily centered around 1000 (see Figure 1).

We now also give these plots for the DNA sequences using the same m as given in Section 2.3.

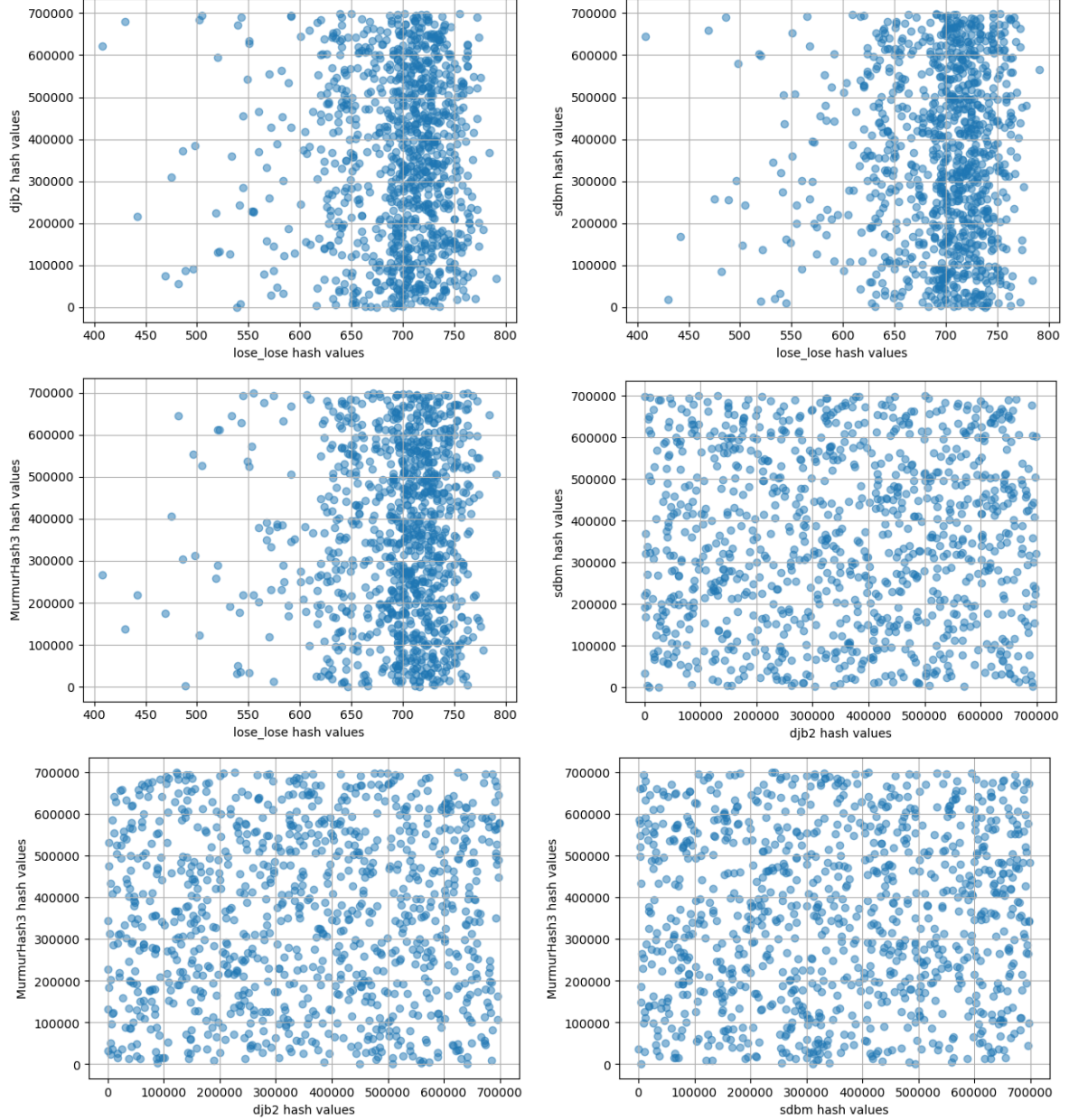


Figure 4: Correlation plots of each hash function combination for a dataset of DNA sequences

In general there does not seem to be any correlation between any of the hash functions. The plots for the `lose_lose` hash function is more squeezed but this is due to the distribution of the hash values being heavily centered around 750 (see Figure 2).

We will not give correlation plots of the g_i functions defined in equation 2 for the Kirsch-Mitzenmacher optimization. This is because [1] proves that if you have two hash functions which are uniformly distributed and independent, that the false positive rate for the filter with functions g_i minimizes to the result acquired in Section 1.2. There is thus no need to analyze the exact structure of the g_i functions.

3 Empirical analysis

3.1 Optimal number of hash functions

We will now compare the theoretical false positive rate against an empirical false positive rate, to test if the minimal false positive rate is reached for $k = \frac{m}{n} \ln 2$. We test this result for our two datasets of English words and DNA sequences using the Kirsch-Mitzenmacher optimization for the sdbm and MurmurHash3 functions. We use these functions since their distribution of hash values are the most uniformly distributed. It is worth noting that the empirical false positive rate is calculated as the ratio of false positives for the words/DNA sequences which are not added to the Bloom filter, we use this definition since most implemented Bloom filters are only designed for specific strings like for example URLs. For English words we take $n = 3 \cdot 10^4$ and $m = 3 \cdot 10^5$, for DNA sequences we take $n = 2 \cdot 10^5$ and $m = 1 \cdot 10^6$. We choose different values for n and m so we can test if the theoretical false positive rate matches for different n and m values.

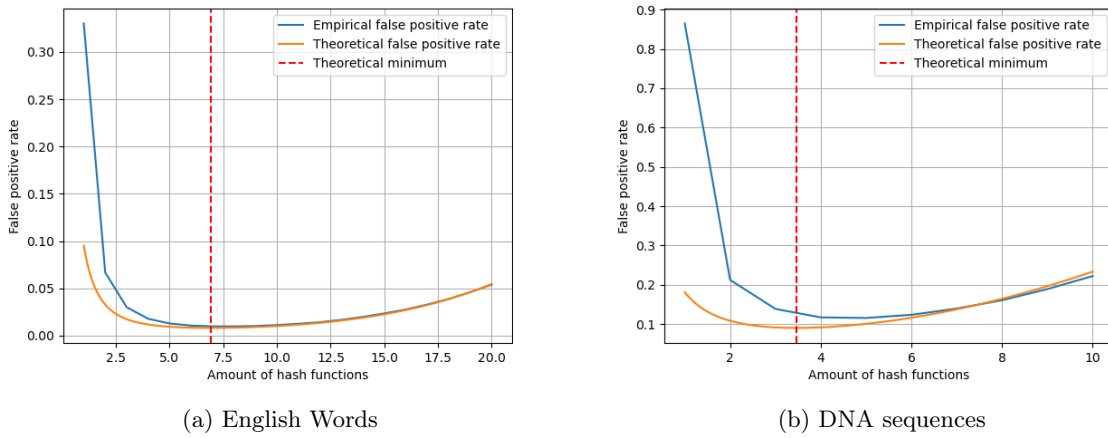


Figure 5: Line plots of theoretical and empirical false positive rates against amount of hash functions for the datasets of English words and DNA sequences.

In general we can see that the optimal value of k ($\frac{m}{n} \ln 2$) gives a low value for the false positive rate but not the exact minimum. We also see that the empirical false positive rate is greatly larger than the theoretical rate for low k values, but that they seem to converge as k increases.

There are a multitude of reasons why the empirical and theoretical false positive rates possibly don't match one to one. First the theoretical false positive rate was an approximation and not an exact equality. Secondly for the derivation of the theoretical false positive rate we had fully independent and uniformly distributed hash functions, in our case the hash functions are mostly uniform, see the bottom plots of Figures 1 and 2, instead of 100 % uniform. There might also be a slight dependence between the k hash functions. Thirdly for the empirical false positive rate we only look at the remaining words/DNA sequences instead of each possible remaining string.

3.2 Convergence of false positive rate

We will now also check the validity of the theoretical false positive rate, given by $\varepsilon \approx (1 - e^{-\frac{kn}{m}})^k$, as our n values increase. We will increase these values to values larger than m to indicate that we add more strings than the amount of strings the Bloom filter was designed for. For the dataset of English words and DNA sequences we will use the sdbm and MurmurHash3 for the Kirsch-Mitzenmacher optimization since these are the most uniform hash functions. For English words we will use a Bloom Filter with $m = 2 \cdot 10^5$ using three hash functions. For the DNA sequences we will use a Bloom Filter with $m = 8 \cdot 10^5$ using five hash functions. This now gives us the following plots in Figure 6.

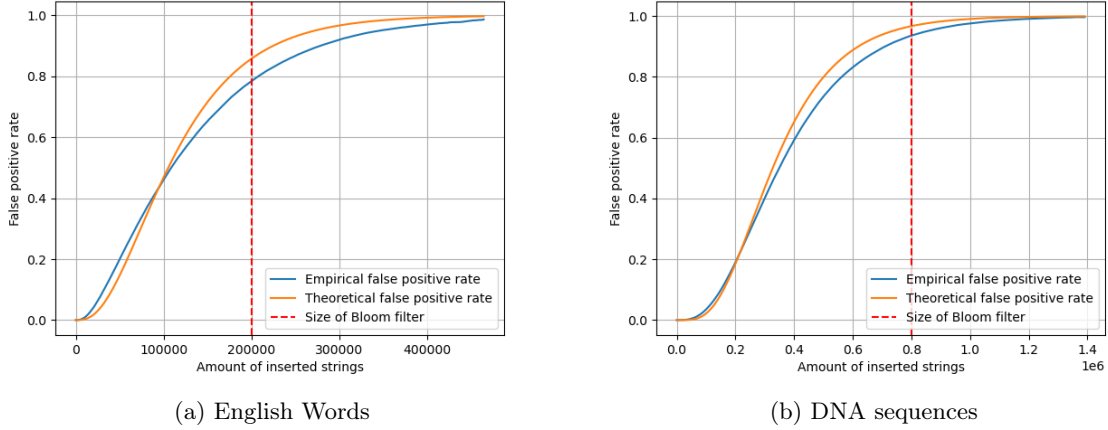


Figure 6: Line plots of theoretical and empirical false positive rates against amount of inserted strings for the datasets of English words and DNA sequences.

In general we see some slight differences between the theoretical and empirical false positive rates, the reasons for this are the same as the reasons given in Section 3.1. In general we see that the false positive rate is very low for small n , but that it rapidly increases as n increases, after that the rate of change slowly decreases and converges to the value of 1. This behavior is expected, for low n values the false positive rate is very low since the probability to have hash values that all correspond to bits with all 'ones' is very low. As n increases the amounts of 'ones' increases which increases the false positive rate drastically (this is also dependent on the amount of hash functions). For very large values of n the bloom filter mostly consists of 'ones' this implies that the false positive rate barely increases since the probability that a new string turns one of the few remaining 'zeroes' into a 'one' is very low.

In general we see that the false positive rate increases drastically as strings are inserted. For example, for a Bloom filter with size m to have a false positive rate less than 10 % we can roughly only insert around $0.10 \cdot m$ to $0.15 \cdot m$ strings. Indicating that if we want to have a false positive rate lower than 10 %, we need the filter size to be roughly ten to fifteen times the amount of strings we wish to insert.

3.3 Effect of increasing filter size on false positive rate

The last variable remaining in our approximation of the false positive rate, given by $\varepsilon \approx (1 - e^{-\frac{kn}{m}})^k$, is the size of the Bloom filter m . We will now analyze how the empirical false positive rate differs from the theoretical rate for a variable value of m . For the English words dataset we will use a Kirsch-Mitzenmacher optimized Bloom filter utilizing three hash functions generated from `sdbm` and `MurmurHash3`. For this filter we will insert $n = 3 \cdot 10^4$ random words. The largest filter which is considered has $m = 1 \cdot 10^6$, this is roughly double the amount of English words in the dataset. For DNA sequences we will use a Kirsch-Mitzenmacher optimized Bloom filter utilizing five hash functions generated from `sdbm` and `MurmurHash3`. For this filter we will insert $n = 2 \cdot 10^5$ random sequences. The largest filter which is considered has $m = 2 \cdot 10^6$, this is roughly double the amount of sequences. This now gives us the following plots in Figure 7

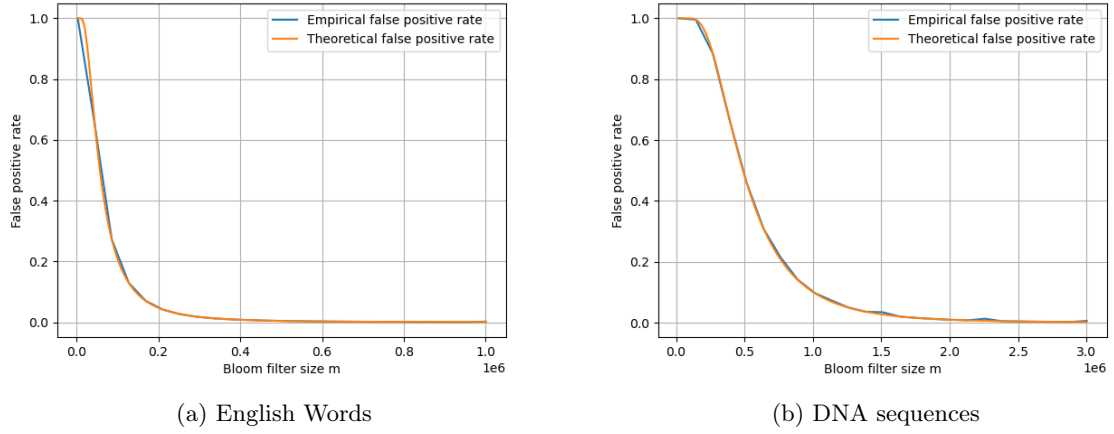


Figure 7: Line plots of theoretical and empirical false positive rates against an increasing filter size m for the datasets of English words and DNA sequences.

From Figure 7 we see no significant deviations between the empirical and theoretical false positive rate.

3.4 Time analysis

3.4.1 Exploratory analysis

We will now look at how much time is needed for adding and searching in the Bloom filter. We will look at two specific Bloom filters, one which fills up very quickly and which contains little information, and one which is of greater size and fills up slowly. The first filter is a Kirsch-Mitzenmacher optimized Bloom filter, with size $m = 2 \cdot 10^5$, using three hash functions constructed from `sdbm` and `MurmurHash3`. This filter will be used on the English words dataset. The ratio between the filter size and the total English words is roughly 0.42 which indicates that the filter fills up very quickly. The second filter is also Kirsch-Mitzenmacher optimized Bloom filter, with $m = 8 \cdot 10^6$, using five hash functions constructed from `sdbm` and `MurmurHash3`. This filter will be used on the DNA sequences dataset. The ratio between the filter size and the total amount of DNA sequences is roughly 5.7, which indicates that the filter fills up very slowly and is mostly sparse. For these filters we will now check the time needed to search and add every string in their corresponding dataset. For these filters we will permanently add these strings to the Bloom filter, since testing the search function for an entirely empty or full filter is of little relevance.

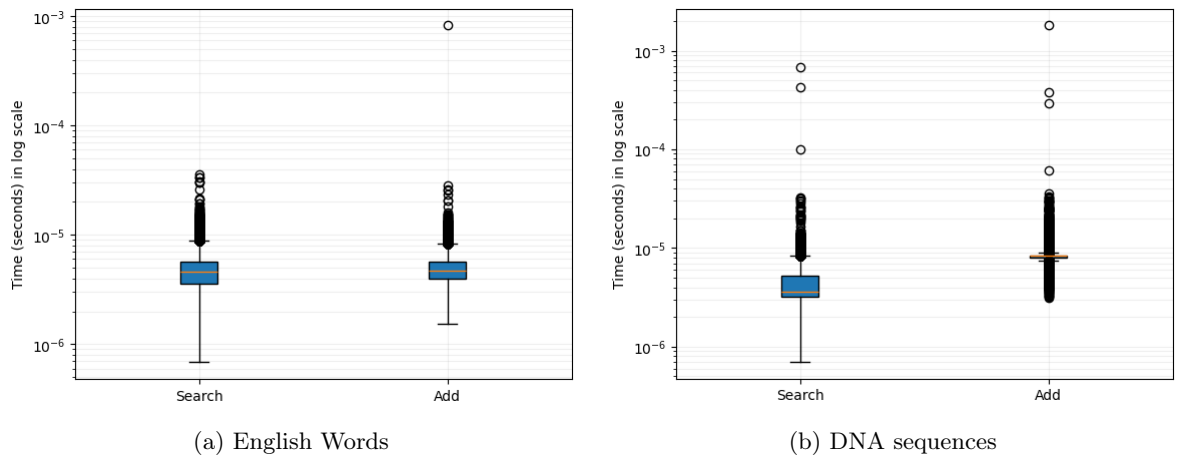


Figure 8: Box plots of the time needed to search and add all of the strings for the datasets of English words and DNA sequences.

An expected result seen in Figure 8 is that the whiskers for search are generally lower than for add. This is expected since if for one of the hash functions the corresponding index is zero the procedure fully stops as compared to adding a string which has to iterate over each hash function. What is surprising is that the medians for search and add for the compact bloom Filter for English words are roughly the same. This can be explained by the fact that the filter fills with ones very quickly which turns requires the search function to iterate over every hash function, since the chance of ending the procedure by reaching a zero is relatively low. The opposite effect of this, thus a lot of zeroes and relatively little ones, can be seen in the sparse filter for DNA sequences, in which the median of search is strictly beneath the box of for adding an element.

We can analyze this effect a bit further by looking at the difference of adding and searching of each string for both of the datasets. These plots are given below in Figure 9.

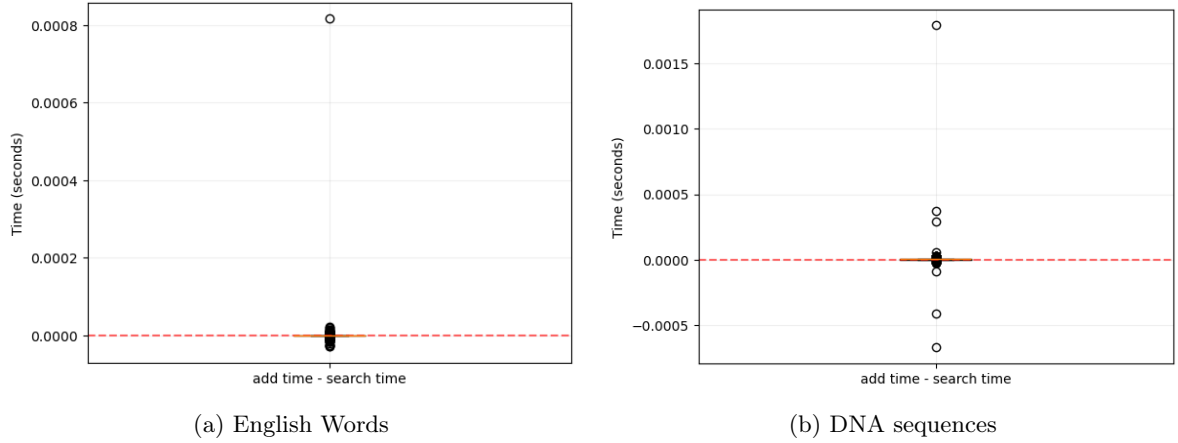


Figure 9: Box plots of the time difference between adding and searching the same string, for each strings for the datasets of English words and DNA sequences.

Since these plots are not very readable due to outliers and since we can't alleviate this since a log scale isn't defined for negative numbers, we give the same box plots but without the outliers in Figure 10.

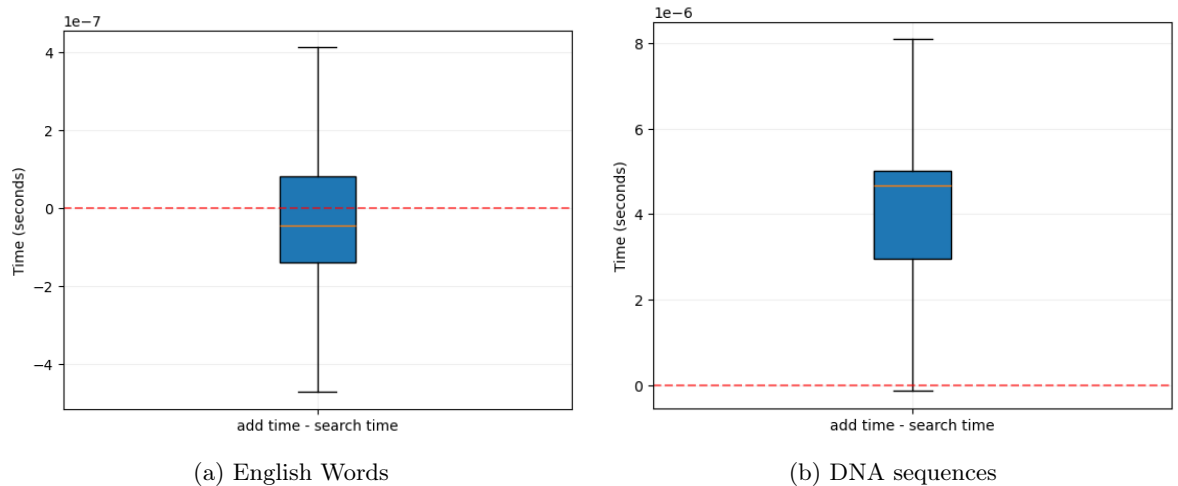


Figure 10: Box plots of the time difference between adding and searching the same string, for each strings for the datasets of English words and DNA sequences, disregarding all outliers.

As expected, we see here for DNA sequences that it takes more time to add an element as compared to search it. What is noteworthy is that there are more English words where it takes more time to search as compared to adding it. This is counterintuitive because even in worst case, where there

are no zeroes, we expect the time to be roughly the same since in both cases we have to iterate over each hash function. A possible explanation is due to the implementation in Python, it might be that checking if a index in the Bloom filter is equal to zero requires more time than modifying that value to be a one.

Besides box plots we can also look at line plots which show the cumulative time for adding and searching as we iterate over each string in the datasets. These are given in Figure 11 given below.

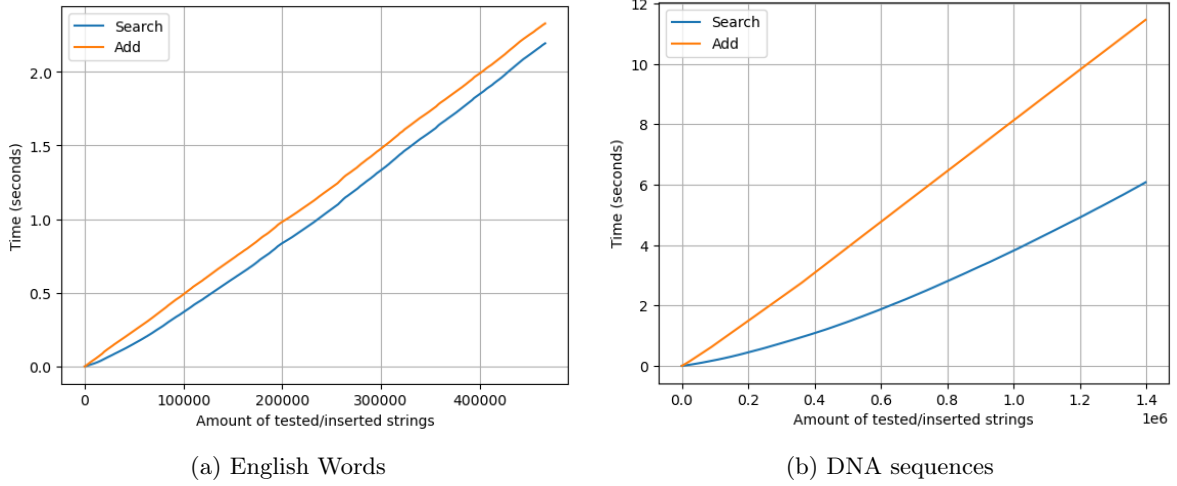


Figure 11: Line plots of the cumulative time needed to search and add for an increasing amount of strings.

These plots reassure our belief that when zooming out / looking at the cumulative time that in both cases searching is faster than adding. For adding we see a linear trend, this makes sense since the addition of a new string is not dependent on the actual array of the bloom Filter. This is not the case for searching since the iteration over the hash functions stops if one of the corresponding indices is one. For English words we see that the difference between searching and adding is relatively minor, this is because in a compact filter there are a lot less zeroes which means that the iteration procedure isn't halted in most cases. The opposite is visible for DNA sequences since there are a lot more zeroes the iteration stops before reaching the end in most cases, which drastically decreases the amount of time needed.

3.4.2 Effect of hash functions

In Section 1.3 we noted that the time complexity of adding and searching, while not increasing string length, is $O(k)$, in which k denotes the amount of hash functions. We are now going to test that assumption for the English words and DNA sequences datasets. For these datasets we will use the same Bloom filters as used in Section 3.4.1, the only deviation being that the amount of hash functions is variable. This is done by adding more functions using the Kirsch-Mitzenmacher optimization. We now get Figure 12, which is given below.

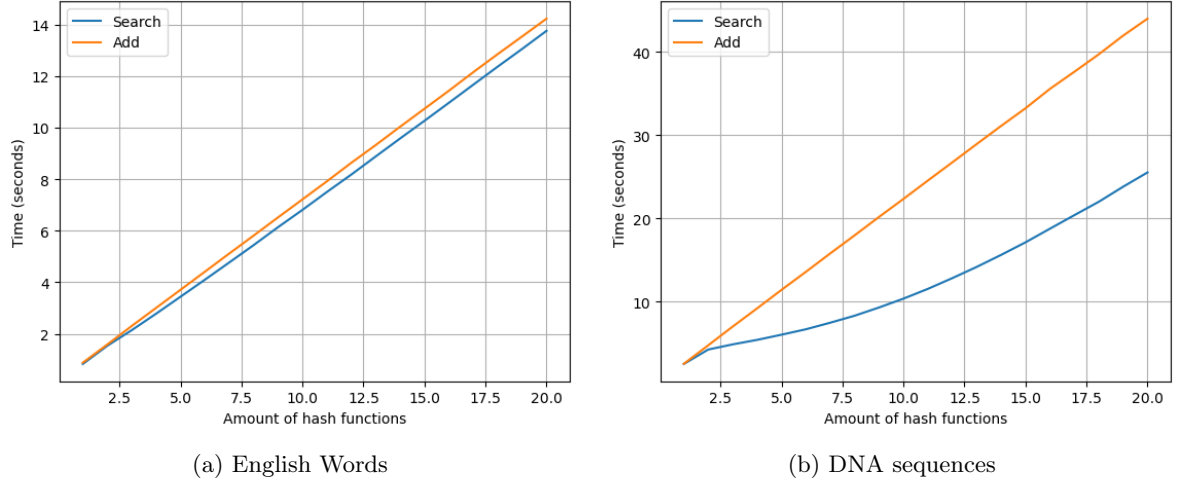


Figure 12: Line plots of the cumulative time needed to search and add for an increasing amount of hash functions constructed using the Kirsch-Mitzenmacher optimization.

From these figures we see that adding shows a predominantly linear trend as the amount of hash functions increase. Indicating that adding does seem to correspond to $O(k)$.

From the figures we also see that searching for English words follows roughly the same trend as adding. This is due to the fact the filter fills up very quickly, which means that for searching we iterate over all of the hash functions, like in adding, as opposed to prematurely ending it when there is an index that corresponds to a zero. For the sparse filter for DNA sequences in which the iteration stops in most cases, we see that searching is better than linear time. Because of this searching does also seem to have an upper bound of $O(k)$.

3.4.3 Effect of increasing string length

We will now analyze the effect of adding strings of an increasing length to a Bloom filter. Since all of our hash functions have a time complexity of $O(l)$, we expect the time complexity of adding and searching an element in a Bloom filter to be given by $O(l)$ under the condition that the amount of hash functions are constant. We will now look at small Bloom filters for which we add strings of the form a , aa , etc. We will first look at a basic filter which only has the lose_lose hash function with a size of $m = 10000$, and for which we also insert strings until a max length of $m = 10000$.

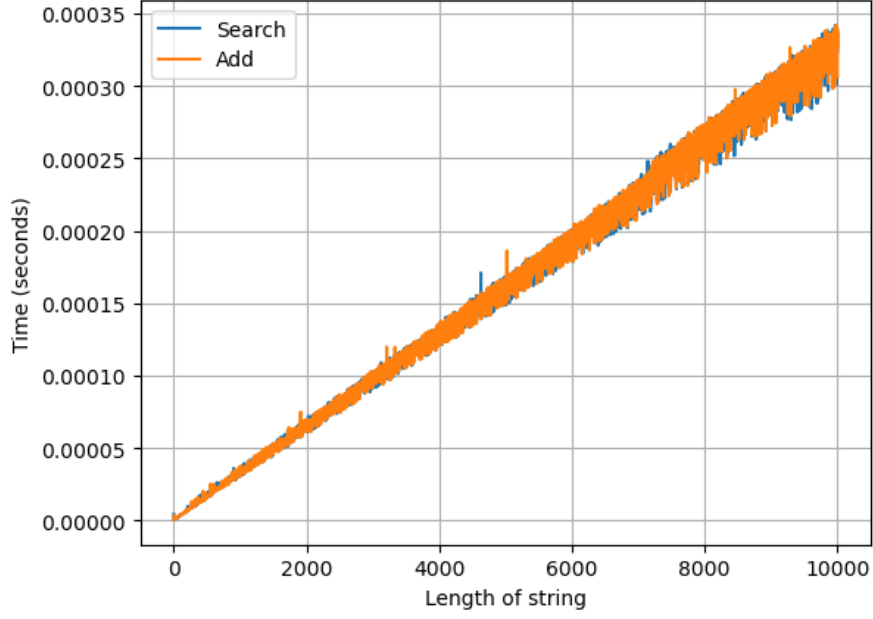


Figure 13: Cumulative time for adding strings of increasing length using the lose_lose hash function.

From Figure 13 we clearly see a linear trend, so it seems that our theoretical results of $O(l)$ is justified. We will now look at a more traditional filter / complicated example using five hash functions generated from the Kirsch-Mitzenmacher optimization from the sdbm and MurmurHash3 functions. We will now add strings of a max length of 1000 to this filter with a size $m = 1000$, this is given in Figure 14.

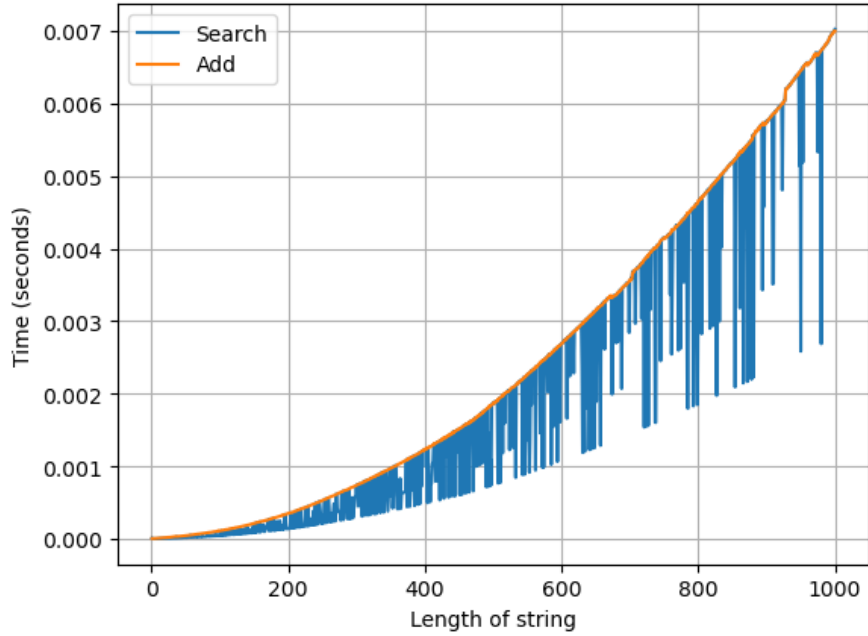


Figure 14: Cumulative time for adding strings of increasing length using five hash functions from the Kirsch-Mitzenmacher optimization for the sdbm and Murmurhash3 functions.

From Figure 14 we see that there is no longer a clear linear trend, we also see that there is greater deviation between the time needed and added. This time discrepancy is due to the fact the filter fills up very quickly using five hash functions and a size of $m = 1000$. The absence of a linear trend can be possibly explained by the fact that the sdbm hash function rapidly increases in value as the amount

of digits increase, this naturally makes the amount of time needed for the addition of values longer.

4 Conclusion

To conclude we showed that the djb2, sdbm and MurmurHash3 functions produce appropriate and approximately uniform values after being modulo reduced by the Bloom filter size m , indicating that these hash functions are suitable for use in Bloom filters. These functions were also tested for correlation, for which there was no indication. Regarding time complexity, under fixed strings, we empirically checked that it was of the form $O(k)$. For increasing string sizes we found a non-linear effect diverging from the theoretical $O(l)$ time complexity. Space complexity is only dependent on the Bloom filter size m and thus was of the form $O(m)$. For a Bloom filter we have a theoretical false positive rate given by: $\varepsilon \approx (1 - e^{-\frac{kn}{m}})^k$, this result was compared to an empirical false positive rate for varying n, m and k values which largely agreed with the theoretical result. For an increasing amount of strings the 'add' and 'search' functions were tested, showing a linear $O(n)$ effect. For 'adding' we also saw an increase of time needed as compared against 'searching', which was to be expected because the searching procedure can stop before iterating over all of the hash functions.

5 Code & Implementation

The plots which are used throughout the report were generated using Python. The Python code, including images of the plots, is given in the following GitHub repository: <https://github.com/KwaZyy/Bloom-filter>. The implementation of the Bloom filter is also explained in the README file of the repository.

6 References

- [1] Adam Kirsch and Michael Mitzenmacher. Less hashing, same performance: Building a better Bloom filter. *Random Structures & Algorithms*, 33(2):187–218, may 15 2008.
- [2] BwandoWando. Dataset containing 479k English words — kaggle.com. <https://www.kaggle.com/datasets/bwadowando/479k-english-words>. [Accessed 25-04-2026].
- [3] Wikipedia contributors. Bloom filter — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Bloom_filter&oldid=1350715730, 2026. [Online; accessed 24-April-2026].
- [4] cse.yorku.ca. <http://www.cse.yorku.ca/~oz/hash.html>. [Accessed 24-04-2026].